

1. 資料怎麼產生？

```
def load_points_from_file(path: str):
    result = []
    with open(path, "r", encoding="utf-8") as f:
        lines = f.readlines()

    # 假設第一行是 header: id A1 A2 A3

    for line in lines[1:]:
        parts = line.strip().split()
        if len(parts) != 4:
            continue
        pid = parts[0]
        x, y, z = map(float, parts[1:])
        result.append((pid, (x, y, z)))
    return result
```

`load_points_from_file` 會把每一行轉成 `(id, (x, y, z))`，存成

`points_with_id`，後面的演算法都用這個 list。

2. 怎麼用 R-tree 產生（建構 3D R-tree）？

MBR + 節點:

```
class MBR3D:
    def __init__(self, low, high):
        self.low = list(low)    # [min_x, min_y, min_z]
        self.high = list(high)  # [max_x, max_y, max_z]

    @staticmethod
    def from_point(p: Point3D):
        return MBR3D(p, p)

    def expand_to_include(self, other: "MBR3D"):
```

```
for i in range(3):
    self.low[i] = min(self.low[i], other.low[i])
    self.high[i] = max(self.high[i], other.high[i])
```

```
class RTreeNode:
    def __init__(self, is_leaf: bool):
        self.is_leaf = is_leaf

        self.children = []    # 葉: [(id, Point3D)] ; 內部: [RTreeNode]

        self.mbr: MBR3D | None = None

    def recompute_mbr(self):
        mbr = None
        if self.is_leaf:
            for _, p in self.children:
                p.mbr = MBR3D.from_point(p)
            ...
        else:
            for child in self.children:
                ...
        self.mbr = mbr
```

MBR3D 負責維護「這個節點涵蓋的最小長方體」，用

expand_to_include 一路往外擴

RTreeNode.recompute_mbr 會看所有子點或子節點，重新算出自己的 **MBR**

建樹（簡化版 bulk-load）：

```
def bulk_load_rtree(points_with_id, max_children=3) -> RTreeNode:
```

```
    # 1. 先依 A1 排序
```

```
    points_sorted = sorted(points_with_id, key=lambda x: x[1][0])
```

```
    # 2. 每 max_children 個點塞成一個葉節點
```

```
    leaves = []
```

```
    for i in range(0, len(points_sorted), max_children):
```

```
        node = RTreeNode(is_leaf=True)
```

```
        node.children = points_sorted[i:i + max_children]
```

```
        node.recompute_mbr()
```

```
        leaves.append(node)
```

```
    # 3. 自底向上 group 成內部節點，直到只剩一個 root
```

```
    level = leaves
```

```
    while len(level) > 1:
```

```
        next_level = []
```

```
        for i in range(0, len(level), max_children):
```

```
            node = RTreeNode(is_leaf=False)
```

```
            node.children = level[i:i + max_children]
```

```
            node.recompute_mbr()
```

```
            next_level.append(node)
```

```
        level = next_level
```

```
    return level[0] # root
```

先把點排序，三個一組裝成葉節點

再把葉節點三個一組裝成上一層內部節點，如此反覆，最後得到

root

每次裝完一個節點就呼叫 `recompute_mbr` 算出對應的包覆框

3. 怎麼走訪 R-tree 產生 skyline (BBS 演算法) ?

關鍵：判斷 MBR/點是否被 skyline 支配：

```
def dominated_by_skyline(p: Point3D, skyline_points: List[Point3D]) -> bool:
```

```
    for s in skyline_points:
        if dominates(s, p):
            return True
    return False
```

```
def mbr_corner_dominated(mbr: MBR3D, skyline_points: List[Point3D]) -> bool:
```

```
    corner = tuple(mbr.low) # 左下角 (min_x, min_y, min_z)
    return dominated_by_skyline(corner, skyline_points)
```

mbr_corner_dominated：如果 MBR 左下角被某個 skyline 點支配，

整個 MBR 裡的點都不可能是 skyline，可以整塊剪掉

BBS 主迴圈：用 heap 走訪並產生 skyline

```
def skyline_with_rtree(root: RTreeNode):
```

```
    skyline_ids = []
```

```
    skyline_points = [] # 只存座標，用來判斷支配
```

```
    # heap 裡放：(mindist, kind, obj)
```

```
    # kind: 0 = node, 1 = point
```

```
    heap = []
```

```

heapq.heappush(heap, (root.mbr.mindist_to_origin(), 0, root))

while heap:
    dist, kind, obj = heapq.heappop(heap)

    if kind == 0:  # 處理「節點」
        node: RTreeNode = obj
        if skyline_points and mbr_corner_dominated(node.mbr,
skyline_points):
            continue  # 整個 MBR 被支配，直接剪枝
        if node.is_leaf:
            # 葉節點：把裡面的「點」丟進 heap
            for pid, p in node.children:
                if not dominated_by_skyline(p, skyline_points):
                    md = math.sqrt(sum(c * c for c in p))
                    heapq.heappush(heap, (md, 1, (pid, p)))
            else:
                # 內部節點：把子節點丟進 heap
                for child in node.children:
                    md = child.mbr.mindist_to_origin()
                    heapq.heappush(heap, (md, 0, child))

        else:  # kind == 1，處理「點」
            pid, p = obj
            if not dominated_by_skyline(p, skyline_points):
                skyline_ids.append((pid, p))
                skyline_points.append(p)

    return skyline_ids

```

heap 初始化：先把 root 丟進去，優先權是

`root.mbr.mindist_to_origin()`。越靠近原點的 MBR 會越早被處理。

處理節點時：

先看有沒有 skyline 點支配它的左下角（mbr_corner_dominated），有就整個跳過（剪枝）。

若是葉節點，就把裡面的「點」算各自的距離、塞進 heap。

若是內部節點，就把子節點的 mindist 算好，塞進 heap。

處理點時：

看它是否被目前已知的 skyline 點支配，沒被支配就加進

skyline_ids 與 skyline_points。

4. 怎麼走訪 SFS 產生 skyline:

```
def sfs_skyline_with_id(points_with_id):
```

```
    # 1. 給每個點計算加總分數（如  $x + y + z$ ），準備排序
```

```
    scored = []
```

```
    for pid, (x, y, z) in points_with_id:
```

```
        score = x + y + z
```

```
        scored.append((score, pid, (x, y, z)))
```

```
    scored.sort(key=lambda t: t[0]) # 2. 將點按照加總分排序，分數
```

越小越有機會進 Skyline

```
    skyline: List[Tuple[str, Point3D]] = []
```

```
    for _, pid, p in scored: # 3. 按順序一個一個檢查每個點
```

```
        dominated = False
```

```

to_remove = []
for idx, (sid, sp) in enumerate(skyline):
    if dominates(sp, p):      # 4-1. 若已在 skyline 的某個
點支配新點，就略過新點
        dominated = True
        break
    if dominates(p, sp):      # 4-2. 反之，新點支配舊的
Skyline 點，標記待移除
        to_remove.append(idx)
if not dominated:
    if to_remove:
        skyline = [item for i, item in enumerate(skyline) if i
not in to_remove] # 5. 移除被新點支配的舊 Skyline 點
        skyline.append((pid, p)) # 6. 新點自身進 Skyline
return skyline

```

排序 → 給每個點計分，從小到大排。

走訪 → 一個個輪流檢查。

過濾 → 新點如果沒被目前 Skyline 支配，就納入 Skyline。反之則丟掉。

移除舊點 → 若新點比現有 Skyline 某些點還強，把它們移出來。

執行結果:

Skyline Query 結果與時間比較

【暴力法 (Brute-force) Skyline】

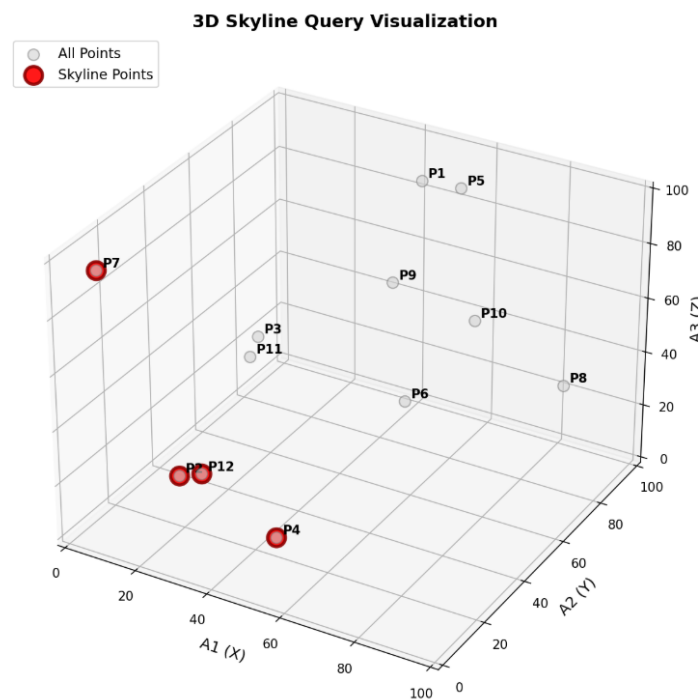
P12: (31, 11, 29)
P2: (11, 32, 5)
P4: (52, 10, 15)
P7: (6, 7, 95)
執行時間: 0.077 ms

【SFS Skyline】

P12: (31, 11, 29)
P2: (11, 32, 5)
P4: (52, 10, 15)
P7: (6, 7, 95)
執行時間: 0.039 ms

【R-tree BBS Skyline】

P12: (31, 11, 29)
P2: (11, 32, 5)
P4: (52, 10, 15)
P7: (6, 7, 95)
執行時間: 0.113 ms



在我們執行一萬筆資料的時候，有出現一個問題

執行結果會顯示: `TypeError: '<' not supported between instances of`

'RTreeNode' and 'RTreeNode'

因為把(mindist, kind, obj) 這個 tuple 丟進 heap，如果前兩格「數值完全一樣」時，heapq 會去比較第三項（obj 物件本身），而你的 RTreeNode 不能互相比較（沒有 < 定義），所以會報錯。

那我們就是加一個「唯一的數字 tiebreaker」，讓這個 tuple「前三項」就保證可以比較，不會比到 obj 物件本身。

在開頭的部分加:

```
import itertools
counter = itertools.count()
```

然後下面每次 heappush，都要補進 next(counter) 這一欄

```
# heap 裡放：(mindist, kind, tiebreaker, obj)

heap = []
heapq.heappush(heap, (root.mbr.mindist_to_origin(), 0, next(counter),
root))

while heap:
    dist, kind, _, obj = heapq.heappop(heap)

    if kind == 0:
        # ...（原本不變）
        if node.is_leaf:
            for pid, p in node.children:
                if not dominated_by_skyline(p, skyline_points):
```

```

        md = math.sqrt(sum(c * c for c in p))
        heapq.heappush(heap, (md, 1, next(counter),
(pid, p)))
    else:
        for child in node.children:
            md = child.mbr.mindist_to_origin()
            heapq.heappush(heap, (md, 0, next(counter), child))
    else:
        pid, p = obj
        if not dominated_by_skyline(p, skyline_points):
            skyline_ids.append((pid, p))
            skyline_points.append(p)

```

heappop 時記得多一個 _ 來吃掉 tiebreaker

執行結果:

1. 讀取 point.txt...
成功讀取 10000 個點
2. 執行 暴力法 (Brute-force) skyline...
完成，skyline 點數: 12，花費 1169.688 ms
3. 執行 SFS (Sort-Filter-Skyline) ...
完成，skyline 點數: 12，花費 19.861 ms
4. 建構 3D R-tree 並執行 BBS skyline...
完成，skyline 點數: 12，花費 45.338 ms

三種查詢演算法比較:

	執行速度(跑一萬筆)
暴力法	1169.688ms
SFS	19.861ms
R-tree + BBS	45.338ms

完整程式碼連結:

https://drive.google.com/drive/folders/1H2uZ2cTSk3Kvr3R7SxWiQF1SWzgNH_O6?usp=sharing